// individual project · programming & methods · berghs

const blog =

"Dailies";

How an AI API writes the summaries on my blog

Elias Grünewald Dykert

elias-blog-xi.vercel.app

röktanke.jpg · AI image

// the blog

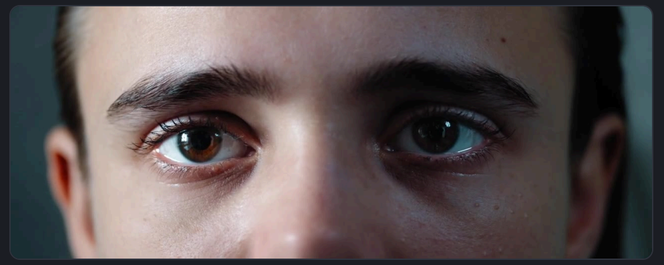
A blog about the process behind my work

name: "Dailies" a film word: the raw, unedited footage you watch after each day of shooting.

about: The process behind my film, AI and art projects, including what didn't work.

for: Classmates, teachers and possibly creative agencies.

posts: 8 posts from my portfolio: Film 4 · AI 3 · Art 1







Live demo

▶ open elias-blog-xi.vercel.app

// 1 · the concept

An AI API lets my code talk to an AI

It’s like ChatGPT, but without the chat window.

// ChatGPT

I type a prompt.
I read the answer.

// my blog

My code sends the prompt.
My code saves the answer.
Automatically, every time I publish.

// API = application programming interface: a way for one program to ask another for something

// 1 · the concept

When I publish, the AI writes a summary



flora · 346 words

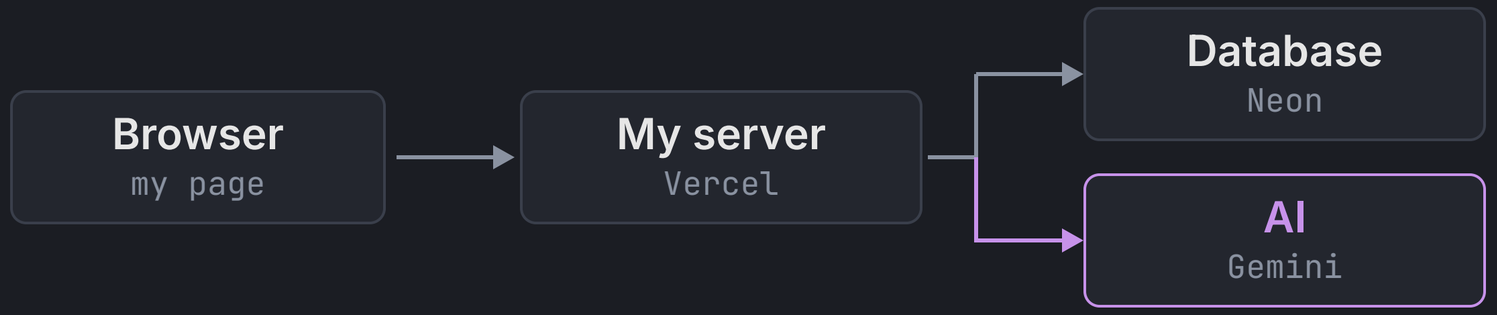
// AI summary · 34 words

Ebba Dankel performs with her trio on an empty concert hall stage for her new music video. The minimalist filming style highlights the physical effort of the music and the connections between the musicians.

written once · saved in the database · labelled “AI summary”

// 2 · how it works

What happens when I click Publish



1 The browser sends my post to the server.

2 The server saves the post in the database.

3 The server sends text and rules to the AI.

4 The AI sends back two sentences.

5 The server saves the summary.

6 The browser shows “saved”.

// the browser never talks to the AI. only my server does.

// 2 · how it works

Three words you need

API key

A secret password that proves the request comes from me. It only lives on the server.

`GEMINI_API_KEY`

Model

Which AI answers. Mine is fast and free.

`gemini-3.5-flash-lite`

Prompt

What I send: my rules plus the post text.

`rules + post`

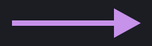
// bonus: temperature = how creative the AI is. I use 0.3: steady, not creative.

// 2 · how it works

What I send, and what I get back

// I send

model	which AI should answer
rules	two sentences, only facts from the post
post	the text I just wrote
temperature	0.3, stay steady



// I get back

Two sentences of text.

My code saves them in the database, next to the post.

// the secret key goes along with the request, from the server only

```
// 3 · my code · runs when I click Publish (shortened)
```

Save first, then ask the AI

```
const user = await requireUser(req, res);
```

Is someone logged in? If not, stop.

```
const post = await createPost(values, user.id);
```

Save the post in the database.

```
const summary = await trySummarize(post.body);
```

Ask the AI. If it fails, the result is empty.

```
if (summary) await updateSummary(post.id, summary);
```

Got a summary? Save it next to the post.

Why this order? If the AI is slow or down, my post is already saved.

// 3 · my code

My rules for the AI, and how it broke them

// my rules

Two sentences, max 45 words

Never “I” or “this post”

Only facts from the post

Nothing else, just the summary

// what went wrong

It started with “This post shows…”
→ I made the rule stricter.

It invented “astronauts” in my film Ackja.
→ I fixed it, and labelled every summary as AI.

A nonsense test post got no real answer.
→ No summary, but the post was still saved.

A prompt is a request, not a guarantee.

// 3 · my choices

Picking a model: three tried, one chosen

`gemini-2.5-flash-lite`

Didn’t work

Retired. Not available to new users.

`gemini-3.5-flash`

Didn’t work

A “thinking” model. It used its whole budget thinking and never answered.

`gemini-3.5-flash-lite`

Chosen

Fast, free, and follows the rules. About one second per summary.

Plan A was Claude through OpenRouter, but their website was down, so I switched to Gemini.

// 3 · my process

How I worked

With AI

I built it with Claude Code. I wrote the brief and made the decisions. It wrote most of the code.

Step by step

One feature at a time, one commit each. Every decision is written down in PROCESS.md.

Problems

OpenRouter was down, so I switched to Gemini. Too many requests at once gave an error, so I waited and tried again.

// docs: the assignment brief, and Google’s Gemini docs for the key, models and limits

// 4 · recap

Four things to remember

1. My code sends text and rules to the AI, and gets text back.
2. The secret key stays on the server.
3. Save first, then ask the AI.
4. Check what comes back, and label it as AI.

// thanks!

questions?